3주차 3일차: ICommand와 RelayCommand — 30분

1. 학습 목표 — 2분

- Command가 실행 동작과 실행 가능 여부를 제공함을 이해합니다.
- CommunityToolkit.Mvvm으로 버튼을 ViewModel의 명령에 연결합니다.

시작 상태는 day10입니다. 이번 시간부터 외부 NuGet 패키지를 사용합니다. 최초 다운로드 대기는 학습 시간에서 제외합니다.

2. 핵심 개념 설명 — 8분

이벤트 방식에서는 버튼이 창의 처리기를 호출했습니다. Command 방식에서는 버튼이 바인딩으로 명령 객체를 찾아 실행을 요청합니다.

ICommand는 Execute(실행), CanExecute(실행 가능 여부), CanExecuteChanged(가능 여부를 다시 확인하라는 알림)를 약속하는 인터페이스입니다. 직접 구현하면 반복 코드가 생기므로 CommunityToolkit.Mvvm의 RelayCommand를 사용합니다. 공식 RelayCommand 안내

이번 코드	의미
new RelayCommand(Run, CanRun)	실행할 메서드와 가능 여부 판단 메서드를 명령으로 묶음
Command="{Binding RunCommand}"	버튼이 ViewModel의 명령을 사용
NotifyCanExecuteChanged()	입력이 바뀌었으니 버튼의 가능 여부 재확인

CanRun은 공백이 아닌 입력이 있을 때 true입니다. Input이 바뀌면 명령에 알림을 보내 빈 입력에서는 버튼이 비활성화됩니다. RelayCommand가 Input 변화를 저절로 알아채는 것은 아닙니다.

Run 뒤에 괄호를 붙이지 않은 이유는 지금 실행하지 않고 "나중에 실행할 메서드"를 전달하기 때문입니다. 람다식과 델리게이트 이론은 깊게 다루지 않습니다.

교재는 NuGet에서 확인한 CommunityToolkit.Mvvm 8.4.2로 버전을 고정합니다. 최신 버전으로 계속 바꾸는 학습은 하지 않습니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
public IRelayCommand RunCommand { get; }

public MainViewModel()
{
    RunCommand = new RelayCommand(Run, CanRun);
}

private bool CanRun()
{
    return !string.IsNullOrWhiteSpace(Input);
}
```

Input의 set에 들어 있는 알림입니다.

```
        input = value;
        Notify(nameof(Input));
        RunCommand.NotifyCanExecuteChanged();
```

버튼의 Click 연결은 제거하고 Command를 연결합니다.

```
<Button Command="{Binding RunCommand}" Content="실행" />
```

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
WpfTestApp.csproj	day11의 WpfTestApp.csproj
MainWindow.xaml	day11의 MainWindow.xaml
MainViewModel.cs	day11의 MainViewModel.cs
MainWindow.xaml.cs	day11의 MainWindow.xaml.cs

4. Visual Studio에서 따라 하는 실습 — 12분

1. **3분:** 실행을 종료합니다. 프로젝트 우클릭 → **NuGet 패키지 관리** → 찾아보기에서 CommunityToolkit.Mvvm을 검색합니다. 버전 8.4.2를 선택하여 설치합니다. 설치 후 csproj에 PackageReference가 있는지 봅니다. 전체 csproj를 교체한 경우에는 솔루션 빌드 시 복원되므로 중복 설치할 필요가 없습니다.
2. **4분:** MainWindow 두 파일과 MainViewModel.cs를 오늘의 전체 코드로 교체합니다. using CommunityToolkit.Mvvm.Input이 추가되고, 창의 클릭 처리기가 제거된 것을 확인합니다. Post.cs는 유지합니다.
3. **3분:** F5로 실행합니다. 기본 입력에서는 버튼이 활성화되고, 모두 지우면 비활성화됩니다. 다시 텍스트를 입력하고 실행해 기존 행 변경이 되는지 확인합니다.
4. **2분:** 입력값 변경 → Input의 set → NotifyCanExecuteChanged → 버튼 상태 재확인 경로를 코드에서 찾고 종료합니다.

패키지를 찾지 못하면 NuGet 원본이 nuget.org인지와 인터넷 연결을 확인합니다. IRelayCommand는 Toolkit이 제공하는 ICommand 확장 인터페이스이며 내부 구현을 암기하지 않습니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 10로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 11로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- Command는 동작과 실행 가능 여부를 화면에 제공합니다.

- RelayCommand는 ICommand 구현의 반복 작업을 줄입니다.
- 입력 변경 시 NotifyCanExecuteChanged를 호출해야 버튼이 다시 판단합니다.
- 클릭을 Command로 연결한 뒤 ViewModel은 버튼을 직접 수정하지 않습니다.

완료본: day11. 다음: 3주차 4일차.